

QA Automation – Azure DevOps

Guia de Utilização — Administrador

Sumário

1. O Papel do Administrador
2. Aprovando Acessos
3. Cadastrando e Editando Usuários
4. Permissões Granulares

4.1 Configurações do app (aba Configurações)

Configurações globais — valem para todos os usuários e só o dono altera. Hoje existe uma: de onde saem as chamadas dos Testes de API.

Navegador do usuário (padrão): as requisições são feitas pelo navegador de quem está usando o app, com o IP da pessoa — o mesmo que funciona no Postman. É o modo necessário para o HML, cujo WAF (CloudFront) bloqueia servidores em nuvem (Streamlit Cloud, n8n). Exige que a API aceite CORS a partir do app.

Servidor do app: as requisições saem do Streamlit Cloud. Use para APIs sem bloqueio de origem ou que não permitam CORS.

💡 A mudança vale para todos imediatamente; quem já estiver logado passa a usar o novo modo ao abrir uma nova sessão. Os usuários não escolhem o modo — só veem uma linha informativa na etapa de execução.

5. Sessões Ativas e Revogação
6. Logs de Auditoria
7. Configuração Inicial (secrets.toml e n8n)
8. Manutenção e Solução de Problemas

1. O Papel do Administrador

O administrador (dono do app, configurado em APP_OWNER_USERNAME no secrets.toml) é o único usuário que entra direto, sem precisar de aprovação. Ele tem acesso a uma área exclusiva — Administração — que não aparece para mais ninguém, organizada em 3 abas, mais uma seção de aprovação sempre visível no topo:

- **Solicitações Pendentes (fora das abas):** aprovar ou negar pedidos de acesso — visível pra você e pra qualquer usuário marcado como aprovador.
-  **Usuários:** cadastro, edição e exclusão de usuários — inclui o status de aprovador e todas as permissões granulares, num único formulário por pessoa.
-  **Sessões Ativas:** todas as sessões abertas no momento, com opção de revogar qualquer uma.
-  **Logs de Auditoria:** histórico dos últimos 500 eventos relevantes do app.

 Todas essas seções ficam dentro do botão "  Administração " na barra lateral — ele só aparece pra você (e a parte de Solicitações Pendentes também aparece pra qualquer aprovador cadastrado, não só pra você).

2. Aprovando Acessos

Toda pessoa que não seja o dono do app precisa ter o acesso aprovado a cada nova sessão — a aprovação não fica valendo para sempre, é consumida no momento do login.

Como aprovar

1. Vá em Administração.
2. A seção "Solicitações Pendentes" aparece no topo, antes das abas, com os pedidos aguardando decisão.
3. Clique em "☒ Aprovar" ou "☐ Negar" para cada solicitação.

 Se você não é o dono mas foi cadastrado como aprovador (veja a seção 3), você também vê e pode agir nessa seção — só não vê o resto da área de Administração.

3. Cadastrando e Editando Usuários

Todo o cadastro de usuário — nome, e-mail, usuário de login, senha, modo de acesso, status de aprovador e permissões — é feito num único formulário por pessoa, na aba "  Usuários". Não existe mais uma tela separada só pra aprovadores: marcar alguém como aprovador é só mais um campo dentro do cadastro do usuário.

1. Vá em Administração →  Usuários.
2. Pra editar alguém que já existe, abra o cartão com o nome da pessoa; pra criar alguém novo, abra "  Cadastrar novo usuário".
3. Preencha Nome, E-mail e Usuário de login (obrigatórios). A Senha é opcional ao editar: deixe em branco pra manter a senha atual da pessoa — assim você mexe em permissões, e-mail ou modo de acesso sem precisar saber nem redefinir a senha dela. Só preencha a senha se quiser realmente trocá-la. (Ao CRIAR um usuário novo, a senha continua obrigatória.)
4. Marque "É aprovador" se a pessoa também deve poder aprovar/negar acessos, e marque as permissões desejadas (veja a seção 4).
5. Clique em "  Salvar Alterações" (ou "  Criar Usuário") — o app sempre mostra um resumo do que vai ser gravado, pra confirmar antes de salvar de verdade.

Um atalho "  Conceder tudo" marca de uma vez acesso direto, aprovador e todas as permissões — útil pra cadastrar rapidamente outro administrador de confiança.

 "  Excluir usuário" (dentro do cartão da pessoa) remove login, aprovações e permissões de uma vez — pede confirmação antes, e não pode ser desfeito.

4. Permissões Granulares

Dez funcionalidades do app são liberadas individualmente, por usuário, marcando os checkboxes correspondentes dentro do cadastro de cada pessoa (aba "  Usuários", seção 3) — quem não tem a permissão específica nem vê o botão ou passo correspondente. A cobertura é total: nada fica liberado só por a pessoa conseguir logar.

Permissão	O que libera
assistente_qa	O assistente de QA em si (Passos 1 a 6: documento, dúvidas, Matriz, Casos, Planos, Download). Sem esta permissão a pessoa NÃO vê o fluxo de documentação.
azure_devops	Passo 7 (Integração – os 3 modos de envio) e Criar Query com IA. Não controla mais a geração de Casos a partir de Work Items no Passo 1 — isso ficou disponível a qualquer usuário logado, sem permissão especial (o controle real é o PAT pessoal de cada um).
execution_report	Relatório de Testes
azure_query	Criar Query com IA
manual_testes	Manual de Testes (UAT)
documentos_armazenados	Documentos Armazenados
mapa_mental	Mapa Mental
criar_bug	Criar Bug (com System Info, Acceptance Criteria, Discussion e evidências em imagem) — também é a permissão que libera criar Bugs no Azure DevOps a partir da etapa Análise e Bugs dos Testes de API
criar_work_item	Criar Work Item (qualquer tipo, campos dinâmicos do processo do projeto, e criação de vários filhos sob um pai)
testes_api	Testes de API: gerar a bateria com IA a partir dos Work Items (ou importar do Postman / criar na tela), executar, ver a análise do resultado (o que é bug da API, bateria, perfil ou regra a confirmar, com quem falar) e gerar as evidências .md/.pdf/.zip

Como conceder ou revogar

As permissões são marcadas e desmarcadas direto no cadastro do usuário (aba "  Usuários", seção 3) — não existe mais uma tela separada por permissão. Marque ou desmarque os checkboxes desejados e clique em "  Salvar Alterações".

 **Importante:** um usuário sem **NENHUMA** permissão consegue logar, mas vê uma tela avisando que nada foi liberado pra ele. E um usuário com apenas uma funcionalidade (ex.: só "Criar Bug") já entra direto naquela área, sem passar pelo assistente. O dono do app sempre tem todas as permissões, independente do cadastro.

5. Sessões Ativas e Revogação

O app usa um identificador de sessão opaco (sem informação legível) na URL — o dado real (quem está logado, até quando) fica guardado no n8n. Isso permite revogar remotamente uma sessão a qualquer momento, mesmo que a pessoa ainda tenha o link salvo em algum lugar.

Como revogar uma sessão

1. Vá em Administração → "🔑 Sessões Ativas".
2. A lista mostra todas as sessões abertas no momento, com usuário e data de início — a sua própria sessão aparece marcada.
3. Clique em "Revogar" na sessão desejada.

💡 Se você revogar a própria sessão a partir dessa tela, você é deslogado imediatamente — é o equivalente a um "sair forçado", útil se você suspeitar que seu link de acesso vazou.

💡 Clicar em "📄 Sair" no rodapé da barra lateral também revoga a sessão automaticamente (não é só um "esquecimento" local) — então, na prática, todo logout normal já é seguro.

6. Logs de Auditoria

Os últimos 500 eventos relevantes do app ficam registrados — visível só para você, em Administração → "📄 Logs de Auditoria".

Eventos registrados incluem, entre outros:

- Login bem-sucedido.
- Aprovação ou negação de acesso.
- Cadastro/remoção de aprovador, e concessão/revogação de permissão.
- Integração com Azure DevOps concluída (qualquer um dos 3 modos do Passo 7).
- Relatório de Testes gerado.
- Geração a partir de Work Items.
- Criação de Query com IA.
- Criação de Bug.
- Criação de Bug a partir dos Testes de API (com o número de anexos).
- Correção da bateria de Testes de API pelas respostas reais.
- Ação em Documentos Armazenados (salvar ou excluir).
- Revogação de sessão.
- Início de uma Nova Análise.

Cada linha traz data/hora, usuário, ação, local no app, e um detalhe livre (ex.: nome do projeto envolvido). Há um filtro por usuário disponível no topo da lista.

💡 *Eventos mais antigos que os últimos 500 são descartados automaticamente — não existe um histórico permanente. Se precisar de retenção mais longa no futuro, isso exigiria um banco de dados dedicado, fora do escopo atual do app.*

7. Configuração Inicial (secrets.toml e n8n)

Esta seção é para quem está configurando o app pela primeira vez, ou adicionando um novo ambiente.

7.1 Credenciais de usuário

No arquivo `.streamlit/secrets.toml` (local) ou nos Secrets do Streamlit Community Cloud (produção):

- **APP_OWNER_USERNAME:** o usuário que é sempre o dono do app, sem precisar de aprovação.
- **[credentials.usernames]:** cada usuário com seu hash bcrypt de senha (nunca senha em texto puro).

💡 Gerar um hash bcrypt: no Python, `bcrypt.hashpw(b"senha", bcrypt.gensalt()).decode()`

7.2 Webhooks do n8n

O app depende de 11 workflows publicados no n8n, cada um com sua própria URL de webhook configurada no `secrets.toml` (ver o README do projeto para a lista completa). Todos usam a mesma credencial Header Auth, configurada com o valor de `N8N_API_KEY`.

7.3 Personal Access Token (pessoal ou compartilhado)

Por padrão, cada usuário informa o próprio PAT, na hora, dentro do app — nunca salvo em disco, só na memória da sessão. Se preferir um PAT único pra todo mundo, configure `AZURE_DEVOPS_PAT` no `secrets.toml`: nesse caso ninguém mais precisa digitar token nenhum, e a rastreabilidade de quem fez o quê passa a vir de uma tag automática (criado-por:<usuário>) em cada Bug/Test Case criado. Cada usuário vê um aviso único na primeira vez que esse modo compartilhado estiver ativo pra ela.

8. Manutenção e Solução de Problemas

Um usuário não consegue mais logar / sessão sempre expira

Confira se a organização/URL do n8n está no ar — a validação de sessão depende de uma chamada ao webhook de Controle de Acesso. Se o n8n estiver fora do ar, o login para de funcionar.

Erros de integração com o Azure DevOps

A causa mais comum é PAT com escopo insuficiente ou expirado — confira se o usuário tem os escopos Work Items (Read & Write) e Test Management (Read & Write), e se o token ainda não expirou.

Modelos de IA desatualizados

Os workflows do n8n fixam versões específicas de modelo por provedor (Gemini, Groq, OpenAI, Mistral). Provedores mudam ou depreciam modelos com alguma frequência — se um workflow começar a falhar sistematicamente num provedor específico, vale checar a versão do modelo configurada nesse node.

Quero remover o acesso de alguém definitivamente

Revogar a permissão e a sessão ativa impede o uso imediato, mas a pessoa ainda pode tentar logar de novo (e cair na tela de aprovação). Para bloquear de vez, remova as credenciais dela do secrets.toml.

Falhas de geração dizendo que "todos os provedores de IA falharam"

Na maioria das vezes é rate limit passageiro num provedor (ex.: Groq TPM) — o app já tenta de novo sozinho: quando uma chamada de IA falha (em qualquer área — análise do documento, imagens, Matriz/Casos/Planos, vínculos, textos do relatório, WIQL, manual ou testes de API), espera cerca de 62s (a janela do rate limit) e tenta a mesma chamada de novo, até 3 vezes, antes de desistir de verdade; entre lotes que deram certo espera só 5s. Se o erro persistir depois disso, aí sim vale investigar o provedor específico citado na mensagem de erro. Nos Testes de API, a mensagem de erro já lista o motivo de cada provedor (ex.: "Gemini: too many requests", "Mistral: 429") — quase sempre é cota por minuto/dia dos planos gratuitos; a solução definitiva é um plano pago em pelo menos um provedor (vale para todos os workflows).

Timeout do proxy na frente do n8n

Se o n8n estiver atrás de Nginx/Nginx Proxy Manager, aumente proxy_read_timeout, proxy_connect_timeout e proxy_send_timeout pra pelo menos 300s — sem isso, o proxy pode cortar a conexão antes do n8n terminar, mesmo quando a IA responderia a tempo (ver n8n_workflows/nginx_docker_timeout.md no repositório).

Catálogos dos Testes de API (rotas e formatos)

Pra cada Base URL, o app guarda no banco (Turso, tabela app_config) dois catálogos compartilhados por todos: as rotas reais da API (chave api_rotas::<host>) e o formato real que ela usa — campos validados, caminhos das respostas, mensagens de erro (chave api_contratos::<host>). Os dois são aprendidos sozinhos a cada execução e vão pra geração com IA. Se a API mudar muito (nova versão), remova a chave correspondente no banco: o app reaprende na próxima execução. Criar Bugs a partir dos Testes de API usa a mesma permissão "Criar Bug".